

Sewershed: technical documentation.

Stack, knowledge, data, and every implementation step — in one place.

If the team only opens one document on Saturday, it should be this one.

Map / UI role — read §2, §3 (Leaflet, GeoJSON), §4, §9, §11. Skim §7 for the file you'll consume.

Data / pipeline role — read §3 (spatial topics), §6, §7, §8 in full. You own `segments_scored.json`.

ML role — read §3 (transfer learning), §6.2, §10. You own the catch-basin condition layer.

Pitch role — read §2, §6.3 (canonical numbers), §12, and the demo contract in §9. You own the words.

Contents

1. System overview	7. Scoring specification
2. Knowledge prerequisites (topic primers)	8. Frontend specification
3. Tech stack specification	9. AI integrations
4. Environment setup	10. Exports
5. Data dictionary	11. Verification & testing
6. Pipeline implementation, step by step	12. Troubleshooting
	13. Appendix: URLs and canonical numbers

1. System overview

Sewershed ranks every still-combined sewer street in Somerville by separation-readiness, so the digging in the \$1.29B regional CSO plan — ~103 committed acres of separation in Somerville's catchments, plus conveyance routes and the city's rolling dig calendar — can be sequenced and bundled from data instead of by hand. Sewershed is the coordination layer riding alongside the hydraulic plan, not a replacement for it. It joins the city's pipe network (which streets still run combined) with Cyvl's street-level condition scan (which streets are cheap and valuable to dig), scores each street 0–100, and outputs an interactive map, a 30-year sequencing time-lapse, and a consultant-ready export package.

Architecturally it is a static-first web app: a one-time join script precomputes everything into a single JSON file, the browser does all ranking and animation client-side, and the only live-network calls are the Claude explainer and the optional image features. This is a deliberate reliability choice — the demo works with no wifi.

Layer	Contents	Runs
Precompute	<code>scripts/join.ts</code> : spatial join + scoring → <code>public/data/segments_scored.json</code>	Once, on a laptop
Client	Leaflet map, ranked list, budget slider, time-lapse, weights panel, detail cards	Browser, offline-capable
Server (thin)	<code>/api/explain</code> (Claude streaming), <code>/api/export</code> (DXF generation)	Vercel functions
ML (parallel track)	Catch-basin condition classifier: Claude auto-labels, small model distills	Once, results merged as a map layer

2. Knowledge prerequisites — topic primers

Each primer: what the thing is, where it appears in this build, and the one trap that costs an hour.

2.1 GeoJSON

The JSON format for geographic data. A file is a `FeatureCollection` of `Feature`s; each `Feature` has a `geometry` (`Point`, `LineString`, or `Polygon` with coordinate arrays in `[longitude, latitude]` order) and a `properties` object holding the attributes. Everything in this project — pipes, streets, catchments, scored segments — moves as GeoJSON.

Trap: coordinate order. GeoJSON is lng,lat; Leaflet's API is lat,lng. Mixing them puts Somerville in the ocean off Antarctica. When the map renders blank, check this first.

2.2 Coordinate reference systems

Coordinates only mean something relative to a reference system. Web data uses WGS84 (EPSG:4326, plain lat/lng degrees); municipal GIS often stores Massachusetts State Plane (feet). Our downloads already requested `outSR=4326`, so every local file is WGS84 — but any data fetched fresh on Saturday must repeat that parameter.

Trap: degrees are not meters. One degree of longitude in Somerville $\approx$ 82 km. Never do distance math on raw coordinates; `turf` handles conversion when you pass `{units: 'meters'}`.

2.3 Spatial joins

Joining two datasets that share no key by using location as the key. We use **constrained nearest-neighbor map matching**, not a naive buffer join: an R-tree spatial index (flatbush) over street centerlines, k-nearest candidate lookup for every pipe and observation, then constraint filtering (max distance: 25 m pipes / 15 m observations; same-catchment consistency) and candidate scoring (distance + street-name similarity — `Cyvl` records carry `address_st`). Below a confidence floor, features are marked orphans rather than force-assigned. Section 6 specifies it precisely.

Trap: the all-pairs explosion. 2,438 pipes $\times$ 2,272 streets $\times$ thousands of points is minutes of compute if done naively. The R-tree turns candidate lookup into $O(\log n)$ — and "nearest" without constraints silently mis-assigns at corners and parallel streets, which is why the catchment and name filters exist.

2.4 Leaflet's rendering model

Leaflet draws a slippy map from tile images (we use free OpenStreetMap tiles) and overlays vector layers on top. `L.geoJSON(data, options)` turns a `FeatureCollection` into a layer; the `style` function returns per-feature color/weight; `onEachFeature` attaches click handlers; `layer.setStyle()` recolors without re-adding. Layer groups give the toggle list.

Trap: re-creating layers on every state change. Adding 2,438 polylines is fast once; doing it on every slider tick freezes the page. Create once, restyle with `setStyle` on updates.

2.5 The four turf.js calls we use

Function	Used for	Note
<code>turf.nearestPointOnLine(line, pt)</code>	Snap pipe midpoint $\rightarrow$ street centerline; returns distance	Run only on bbox-prefiltered candidates

<code>turf.booleanPointInPolygon(pt, poly)</code>	Catchment tagging; observation-in-corridor tests	Fast; safe to call in bulk
<code>flatbush</code> R-tree (not turf)	Index street bboxes once; k-NN candidate lookup for every match	Build once (~2,272 entries); replaces buffer+point-in-polygon entirely
<code>turf.length(line, {units: 'miles'})</code>	Segment lengths for costs and stats	—

2.6 Multi-criteria decision analysis (MCDA)

The formal name for what the score does: normalize incommensurable factors onto a common 0–100 scale, then combine with explicit weights. The defensibility comes from transparency — anyone can inspect the factors, the normalizations, and the weights, and change them. Use the term in Q&A; operations-minded judges recognize it.

Trap: normalizing with min/max of the current dataset makes scores unstable when data changes. Use fixed domain anchors (e.g., pavement 0–100 already; age anchored 1870–2026).

2.7 Greedy selection and the knapsack frame

The budget slider answers "which streets fit in \$X?" — formally a knapsack problem, which is NP-hard. We use the standard greedy approximation: sort by readiness-per-dollar, take from the top until the budget is spent. For segments of similar cost the greedy result is near-optimal, and unlike an optimizer it can be explained in one sentence on stage.

2.8 Transfer learning and distillation

Pretrained vision models already encode what grates, leaves, asphalt, and metal look like. *Transfer learning*: reuse that encoding and train only a small classification head on ~150 labels. *Distillation*: a frontier model (Claude vision) generates the labels; a small cheap model learns them and runs at city scale for free. Humans audit a sample rather than labeling everything — auto-label, human-audit, distill.

Trap: class imbalance. Random catch basins are ~85% "clear." Enrich rare classes when gathering crops, or the model learns to always answer "clear."

2.9 MCP in one paragraph

The Model Context Protocol standardizes how AI clients call external tools. The Cyvl MCP server exposes their data API as tools that Claude Code can invoke directly — meaning during the build, you can ask Claude Code "pull pavement scores for this bbox" and it writes and runs the query itself. Connection is already verified; project id is in §6.3.

2.10 DXF

Autodesk's drawing-exchange format: plain text describing CAD entities on named layers. We emit ranked street polylines (layer per rank tier, e.g. `DIG_TIER_1`) plus text labels via the `dxw-writer` npm package, and pair the drawing with a CSV attribute schedule. A drawing plus a schedule is what enters a consultant's Civil 3D workflow — call it a package, not a capital plan.

3. Tech stack specification

Package	Version	Role	Why this one
next	15.x (App Router)	App framework + API routes	One repo, one deploy; server routes only where needed
react / react-dom	19.x	UI	Ships with Next 15
leaflet + react-leaflet	1.9.x / 5.x	Map	No token, no billing, OSM tiles; what the reference build used
@turf/turf	7.x	Geospatial math	All four needed operations, pure JS, runs in a script
zustand	5.x	State	One store: scenario, weights, selection; no boilerplate
ai (Vercel AI SDK) + @anthropic-ai/sdk	latest	Claude explainer streaming	\$200 team credits; streaming in ~20 lines
dxf-writer	1.x	DXF export	Simple polyline/text emission; no CAD dependency
tailwindcss	4.x	Styling	Ships with create-next-app; fast polish
ML track: Python 3.11+, torch, transformers or open_clip, scikit-learn	—	Classifier	CLIP features + logistic head (fast path) or ViT fine-tune (GPU path)

3.1 Deliberately rejected, and why

Rejected	Reason
Mapbox GL	Token + billing risk on event wifi; Leaflet covers every needed interaction
PostGIS / any database	The dataset fits in one JSON file; a DB adds an hour of ops for zero demo value
Python geospatial service (geopandas/shapely)	Two runtimes to deploy; turf in a Node script does the same join
Vector DB / custom embeddings	Cyvl already hosts embeddings search; corpus too small to justify RAG anywhere else
Hydraulic modeling (SWMM)	Out of scope by design — we sequence, we don't simulate flow

4. Environment setup (exact commands)

```
# 1. Scaffold (Saturday 9:05)
npx create-next-app@latest sewershed --typescript --tailwind --app --no-src-dir
cd sewershed

# 2. Dependencies
npm i leaflet react-leaflet @turf/turf flatbush zustand ai @anthropic-ai/sdk dxf-writer
npm i -D @types/leaflet tsx

# 3. Data in
cp -R ~/Desktop/cyvl_hackathon/data public/data

# 4. Deploy pipeline
git init && git add -A && git commit -m "scaffold"
gh repo create sewershed --private --source . --push
npx vercel link && npx vercel --prod # hello-world URL exists by 9:25
```

4.1 Environment variables

Variable	Source	Available
ANTHROPIC_API_KEY	Team key at check-in (\$200)	Sat 9:00
CYVL_API_KEY	Check-in	Sat 9:00
CYVL_PROJECT_ID	f15b854a-d203-49c7-bc25-1350dd4a1cd6 (verified via MCP)	now

Set in Vercel dashboard → pull locally with `npx vercel env pull .env.local`. Keys never enter git; `.env*` is already in the scaffold's `.gitignore` — verify before first push.

4.2 Repo layout (target state)

```
sewershed/
├─ app/
│  └─ page.tsx # map + panels (client component shell)
│     └─ api/ explain/route.ts · export/route.ts
├─ components/ MapView · BudgetSlider · TimeLapse · SegmentCard · RankedList · WeightsPanel · Layer
├─ lib/ score.ts · greedy.ts · store.ts · cyvl.ts
├─ scripts/ join.ts # the precompute – run with: npx tsx scripts/join.ts
├─ ml/ gather.py · label.py · train.py · predict.py (ML track, parallel)
└─ public/data/ *.geojson · segments_scored.json · basin_conditions.json
```

5. Data dictionary

5.1 Local city GIS files (`public/data/` , all WGS84, all verified by recount)

File	Features	Fields we use	Known defects (verified)
sanitary_mains	4,447 LineString	WATERTYPE ("Combined": 2,438 / "Sewage": 1,934), INSTALLDAT , LINEDYEAR , FROMMH / TOMH	INSTALLDAT only 30% populated (1,346); oldest 1800, 310 pre-1930
storm_mains	1,983 LineString	WaterType , geometry	field name casing differs from sanitary file
sewer_subsystems	7 Polygon	TRIB_NAME , SumTribIn (A, CA, C1, C2, S1, S2, M)	—
ss_manholes / sw_manholes	3,435 / 1,521 Point	location only (density input)	CUTDEPTH 0% populated; rim/invert 9 of 3,435 — depth is unusable, by design we exclude it
storm_inlets	3,659 Point	location (drainage density)	server ignores resultOffset — paginate by OBJECTID
storm_discharge_points	27 Point	PERMIT / PERMITID	—
street_centerlines	2,272 LineString	Street , FuncClass , Ownership	—
sidewalks	3,078 Polygon	Last_SCI (98% populated), Constr_Dat , material %	—
mwra_sewer_lines / nodes	275 / 271	context layer only	—

5.2 Cyvl API surface (base `i3.cyvl.app` , Bearer auth; same data via MCP)

Endpoint	Key params	Feeds
GET <code>/api/v1/pavement/scores</code>	<code>project_id</code> , <code>bbox</code> , <code>score_min/max</code> , cursor pagination	Dig-once factor (30 ft line geometry + score)
GET <code>/api/v1/pavement/distresses</code>	<code>distress_type=</code> utility-cut patching value from <code>/reference/distress-types</code>	Congestion factor
GET <code>/api/v1/assets</code>	<code>asset_type=CATCH_BASIN,MANHOLE_COVER,RAMP...</code>	Density + bundling; each record carries <code>image_url</code>
GET <code>/api/v1/assets/{id}/imagery</code>	—	Detail-card photos
POST <code>/api/v1/embeddings/query</code>	text query, <code>min_score</code> , <code>project_id</code>	Live search demo beat — run against "Boston Full" project <code>`8d8f8cd6-f25a-470c-</code>

		88fd-6b0e0ad4d1d7` (236,983 embedded images, verified working); Somerville project has 0 embeddings
Somerville ArcGIS (no auth)	<code>/{{layer}}/query? where=1=1&outFields=*&f=geojson ; layers 11/12/24 need f=json&outSR=4326 + OBJECTID paging</code>	Already downloaded; re-fetch only if needed

5.3 Canonical numbers (single source of truth — never improvise others)

Program total	\$1.29B = Alewife \$340M + Mystic \$260M + Charles \$690M (draft plan Table 1)
Combined segments	2,438 = 55% of the <i>sanitary</i> network (38% of all mains — always say "sanitary")
Combined pipe length	71.7 miles (computed from geometry)
Committed separation (Somerville catchments)	103 acres = Alewife 8 + Mystic 95 (plan Table 1) $\approx$ 21 street-miles. The Charles' 446 acres are Cambridge-side. The plan chose tanks/tunnels over full separation — never claim "we sequence separating all 2,438"
Savings envelope	\$5–10M modeled on the committed separation alone $\approx$ 7–10% of the street-work segment ($\approx$ \$70–100M, our estimate; tanks/tunnels consume the rest of the \$600M Somerville-watershed share). Mechanism = utility cuts shorten pavement life (LA 25→16.5 yrs; Phoenix –4.5 yrs). Never cite McKinsey
Poor-or-worse pavement	940 segments (from Cylv PCI distribution via MCP)
Claim discipline	"No <i>public</i> tool" (not "zero tools") · 311 clustering is a <i>test</i> , not a result, until run · no CSO gallons per street, ever

6. Pipeline implementation, step by step

One script, `scripts/join.ts`, run with `npx tsx scripts/join.ts`. Target runtime: under 2 minutes. Process one catchment at a time so partial results exist from the first run.

Step 1 — Load and validate

```
const pipes    = load('sanitary_mains');    // assert 4447
const streets  = load('street_centerlines'); // assert 2272
const basins   = load('sewer_subsystems');  // assert 7
// sanity: first coordinate must be ~[-71.1, 42.4]; if not, projection bug – stop.
```

Step 2 — Filter targets

```
const combined = pipes.filter(f =>
  (f.properties.WATERTYPE ?? '').trim() === 'Combined'); // assert 2438
```

Step 3 — Build the R-tree

Index all 2,272 street-centerline bboxes into a `flatbush` R-tree once (milliseconds). Every subsequent match is an $O(\log n)$ k-nearest query instead of a scan — the difference between seconds and minutes, and the proper data structure to name in Q&A.

Step 4 — Match pipes to streets (constrained k-NN)

```
const tree = buildFlatbush(streets); // once
for (const pipe of combined) {
  const mid = midpointOf(pipe);
  const cands = tree.neighbors(mid, k=5) // R-tree k-NN
    .filter(st => dist(st, mid) < 25) // max-distance constraint
    .filter(st => st.catchment === pipe.catchment); // consistency constraint
  const best = maxBy(cands, st =>
    -dist(st, mid) + 10 * nameSimilarity(st.Street, pipe.street_hint ?? ''));
  best ? assign(pipe, best) : orphans.push(pipe); // never force-assign
}
```

Same matcher for Cyvl observations → streets (15 m threshold; pavement records carry `address_st`, so the name-similarity term does real work there). Expect a small orphan set (easements, parks). Report the count honestly.

Step 5 — Tag catchments (3-point majority)

```
// midpoint alone fails on boundary-crossing streets – vote with 3 samples
const pts = [startOf(street), midpointOf(street), endOf(street)];
street.catchment = majority(pts.map(p =>
  basins.find(b => turf.booleanPointInPolygon(p, b))?.properties.SumTribIn));
```

Step 6 — Attach observations

Run the same constrained k-NN matcher for every observation type (15 m threshold, catchment + name constraints). Parallel streets and corner ambiguity are handled by construction — the constraints decide, not corridor overlap. Distresses skip geometry entirely: they join by `inspect_id` to their pavement record.

Attach	Aggregate to
Cyvl 30 ft pavement scores (midpoints)	pavementScore = mean; pavementN = count
Utility-cut patching distress features	patchCount , patchPerMile
Manholes + utility poles + hydrants (city + Cyvl)	utilityAssetDensity per mile
Sidewalk polygons	sidewalkSCI = length-weighted mean of Last_SCI
Ramps (Cyvl)	rampCount ; intersections without ramps → rampGaps
Pipe attributes	installYear (earliest known), dateKnown flag, linedYear , pipeLengthFt

Step 7 — Score and emit

```
// output: public/data/segments_scored.json
{ "generated": "...", "weights": {...}, "segments": [{
  "id": "st-0412", "street": "...", "from": "...", "to": "...",
  "catchment": "A", "geometry": {...LineString...},
  "factors": { "digOnce": 84, "pipeAge": 71, "congestion": 62,
    "bundling": 58, "leverage": 40 },
  "raw": { "pavementScore": 38, "installYear": 1924, "dateKnown": true,
    "patchCount": 6, "utilityAssetDensity": 41.2,
    "sidewalkSCI": 41, "rampGaps": 2, "pipeLengthFt": 410 },
  "readiness": 91, "estCostM": 1.4, "photoUrl": "https://...cloudfront...jpg"
}]}
```

Fallback (pre-decided): if snapping is still misbehaving at 11:45, switch to majority-bbox-overlap assignment, note it in the README, move on. Plausible attribution beats survey-grade precision for a ranking demo.

7. Scoring specification (judge-corrected)

Factor	Wt	Normalization (→ 0–100)	Missing-data policy
Dig-once urgency	30%	$100 - \text{pavementScore}$; if score ≥ 85 (fresh pavement) apply moratorium penalty: factor = 0	No Cyvl coverage → 50 + "no scan" flag
Pipe age & lining	25%	anchor 1870→100, 2026→0, linear; never-lined +10 (cap 100)	70% undated → median-old (≈ 65) + visible "date unknown" flag; never display a fabricated year
Subsurface congestion (cost proxy)	20%	$100 - \text{minmax}(\text{patchPerMile} + \text{utilityAssetDensity})$ with fixed anchors (0 → 100; $\geq 60/\text{mi}$ → 0)	Depth deliberately excluded — city depth fields verified empty
Bundling value	15%	$0.6 \cdot (100 - \text{sidewalkSCI}) + 0.4 \cdot \min(\text{rampGaps}, 5) \cdot 20$	SCI missing (2%) → 50
Network leverage	10%	share of street's neighbors (via FROMMH/TOMH graph) already separated $\times 100$	topology gap → 0

Contract: `readiness()` is a pure function in `lib/score.ts`, unit-tested with three hand-made segments before integration. Ship with a **sensitivity check**: perturb each weight $\pm 10\%$ and report top-10 stability — "the ranking is robust to the weights" is the strongest possible answer to "aren't your weights arbitrary?". Weights live in the Zustand store; the `WeightsPanel` writes them; re-ranking is recomputation over precomputed factors — no geometry math in the browser, ever.

Cost estimate per segment (for the slider and savings counter): $\text{estCostM} = \text{pipeLengthFt} \times \$1,800/\text{ft} \div 1e6$, the assumption stated on screen. Savings counter: \$0.5M per coordinated street-mile, repaving-overlap basis.

8. Frontend specification

State (Zustand, lib/store.ts)

```
{ catchment: 'A' | ... | 'ALL', budgetM: 10, weights: {...5 keys},
  selectedId: string|null, year: 2026, playing: false,
  layers: { combined: true, separated: true, complaints: false, basins: false } }
```

Component	Behavior contract
MapView	Leaflet with preferCanvas: true (6,400 polylines as canvas strokes, not DOM nodes — smooth pan and time-lapse). Renders once: combined (red), separated (green), catchments (dashed); restyles via setStyle . Click → selectedId ; top-10 get numbered DivIcon markers.
RankedList	Segments of active catchment sorted by readiness; row click ↔ map selection (two-way).
SegmentCard	Photo (the record's photoUrl), factor bars from factors , raw values with flags ("date unknown"), bundling chips, streamed explainer text.
BudgetSlider	Greedy: sort by readiness/estCostM , accumulate until budget; selected set gets highlight style + totals (streets, \$, miles).
TimeLapse	setInterval ~150 ms/tick; each tick flips next ranked street green, maps list-index → year 2026–2056, adds 0.5 × lengthMi M\$ to the counter. Pause/play/reset. Pure client loop — must run offline.
WeightsPanel	5 sliders normalized to sum 100; writes store; rank order animates on change.

Demo contract: every interaction above runs from segments_scored.json with the network cable pulled. The only two live calls (explainer, DXF) have screen-recording backups.

9. AI integrations

9.1 `/api/explain` — the streamed justification

```
system: You are a public-works analyst. Given one street segment's scoring
factors and raw values, write exactly two plain sentences explaining its rank.
Mention only values provided. If dateKnown is false, say the install date is
unrecorded. No marketing language.

user: {"street":"Cedar St","rank":1,"readiness":91,
      "raw":{"pavementScore":38,"installYear":1924,"dateKnown":true,
            "patchCount":6,"sidewalkSCI":41,"rampGaps":2}}
```

Implementation: AI SDK `streamText` with `claude-sonnet-4-6`, `maxOutputTokens: 150`; cache responses in a module map keyed by segment id (same segment → zero cost). Budget: ~\$0.01/explanation; the whole day costs under \$5 of the \$200.

9.2 Distillation pipeline (ML track, `ml/`)

- Gather** — list all 381 catch basins via API/MCP; each record carries `image_url`; download all (~5 min). Embeddings search runs on the Boston Full project (verified, 236,983 images) — Somerville has none, so rare-class enrichment uses Boston imagery while labels/training stay on Somerville's per-asset photos.
- Auto-label** — Claude vision, batched 10 images/call: classes *clear* (openings ≤50% obstructed; shadows/wet ok), *clogged* (>50% debris), *damaged* (broken/missing/sunken grate), *uncertain* (occluded/blurry — excluded from training). Cost ≈ \$2–4.
- Audit** — 4 people verify the same 20 labels (5 min); record agreement %; it goes on a slide. Below 80% → tighten definitions, relabel.
- Train** — fast path: CLIP/DINOv2 frozen features + logistic regression (`scikit-learn`, seconds, CPU). GPU path (NVIDIA story): fine-tune `vit-base` via `transformers Trainer`, 5 epochs. Either way: 80/20 split, quote held-out accuracy only, and apply an **abstain threshold** — predictions below confidence render as "unverified," never a confident wrong color on the map.
- Predict + merge** — score all basins → `public/data/basin_conditions.json` → map layer toggle. Demo line: "Cylv knows where these 381 are; now it knows which are failing." Show 2 misclassifications honestly.

10. Exports (`/api/export`)

Format	Contents
CSV	rank, street, from/to, catchment, readiness, all 5 factors, estCostM, lengthFt, flags (date-unknown, no-scan)
GeoJSON	selected segments with full properties — drops into QGIS/ArcGIS
DXF	<code>dxf-writer</code> : polylines on layers <code>DIG_TIER_1...3</code> (rank 1–10 / 11–30 / rest), text labels "#1 Cedar St (91)"; local coordinate output documented in README

Flow on stage: Export → DXF downloads → drag into viewer.autodesk.com (account created and tested beforehand) → "this package — drawing plus attribute schedule — is what the engineering consultant receives."

11. Verification & testing

When	Check	Pass criterion
9:35	Re-derive headline stats via API (one call each)	Patching $\approx$ 46.5%, PCI distribution matches MCP overview — stats become API-traceable
After Step 2	Combined count	Exactly 2,438 or stop
After Step 4	Orphan rate	< 8% unassigned; eyeball 5 random assignments on geojson.io
After Step 7	Face validity	Top 10 are old/poor-pavement streets; no fresh-pavement street in top 20
14:30	311 clustering test	Complaint density on top-quartile streets vs bottom quartile; report whichever way it lands — it is a test, not a foregone result
16:00	Offline drill	Wifi off: map, list, slider, time-lapse, card (minus explainer) all work
16:30	ML honesty	Held-out accuracy quoted; 2 failure cases in the deck

12. Troubleshooting

Symptom	Likely cause → fix
Map renders ocean / blank	lat/lng order swapped — GeoJSON is lng,lat; Leaflet wants lat,lng
Features plot but offset/distorted	State-plane data fetched without <code>outSR=4326</code>
Cyvl 401/403	Bearer header missing/typo'd; key from check-in not the platform login
Cyvl results truncated	cursor pagination — follow <code>has_more</code> until false
ArcGIS layer truncated at 1,000	layers ignore resultOffset — paginate by <code>where=OBJECTID>last</code>
geojson f= error on ArcGIS	layers 11/12/24 reject <code>f=geojson</code> — use <code>f=json&outSR=4326</code> , convert
Join takes minutes	R-tree not built / rebuilt inside the loop — build flatbush once, query many
Slider/timelapse janky	layers being recreated per tick — create once, <code>setStyle</code> only
Embeddings search 4xx/empty	<code>embedding_count=0</code> on project — ask staff; use per-asset <code>image_url</code> path; demo fallback = classifier layer reveal
CloudFront images blocked in browser	likely referrer/CORS — proxy through <code>/api</code> route or download at build time
Anthropic rate limit during demo	explainer cache means repeats are free; pre-warm top 10 during rehearsal

13. Appendix — URLs and glossary

Cyvl API docs / spec	<code>i3.cyvl.app/docs</code> · <code>i3.cyvl.app/openapi.json</code>
Somerville sewer GIS	<code>maps.somervillema.gov/arcgis/rest/services/SewerAndStormWaterSystemWoodard/MapServer</code>
Somerville 311 (SODA)	<code>data.somervillema.gov</code>
Autodesk Viewer	<code>viewer.autodesk.com</code> (test login before event)
Ask Boston / reference build	<code>askboston.ai</code> · <code>phi-cyvl.github.io</code>
Debug map	<code>geojson.io</code> — drag any output file in

Glossary one-liners: **CSO** combined sewer overflow · **separation** one pipe → two · **sewershed/catchment** area draining to one outfall · **PCI** pavement condition index 0–100 · **SCI** the city's sidewalk condition index · **MCDA** weighted multi-factor scoring · **distillation** frontier model labels, small model learns · **dig once** coordinate trenching with repaving.

Rules compliance: this document specifies Saturday's implementation. No project code exists before kickoff; everything above is specification, not construction.